

Raptors: NumPy functionality, rebuilt in Rust

Goal

Provide NumPy's public functionality through import raptors as np, with a sound memory model and measured performance improvements. Scripts, notebooks, scientific computing, data processing, and services are all within the intended audience.

Current status

The repository contains an experimental legacy prototype. Full compatibility, memory safety, and a speed advantage have not been established. The rebuild plan has been written; implementation gates are pending.

Why rebuild

The previous approach declared broad completion without proving behavior. The new effort starts with a pinned NumPy reference, direct behavioral comparisons, and reviewed storage and mutation invariants.

Execution plan

1. Baseline the prototype and inventory the public API.
2. Build a differential harness against NumPy.
3. Prove checked storage, shared views, and controlled mutation.
4. Complete one numeric path through Python.
5. Demonstrate reproducible performance wins.
6. Expand to full public functionality and validate release wheels.

Evidence before claims

Compatibility requires matching values, types, mutation, errors, and warnings. Safety requires sound invariants and reviewed unsafe boundaries. Performance requires equivalent Python workloads with timing, memory, and regression results.

Scope

Changing the import is acceptable. Replacing NumPy's binary interface underneath compiled extensions is not a release requirement; explicit adapters will support consumers requiring NumPy arrays.

Async job APIs, GPU support, JIT compilation, and distributed execution are deferred. Early releases must identify their supported subset; full public functionality remains the destination.

Next milestone

Deliver the pinned compatibility contract, reproducible baseline, differential harness, and storage prototype before expanding feature coverage.

Detailed plan and acceptance gates: docs/REBUILD_PLAN.md